

MESS MEMBER MANAGEMENT SYSTEM

AIUB Python Mid-Term Project Report | Summer 25-26

1. Project & Member Information

Project Title:	Mess Member Management System (Tkinter + NumPy)
Course Name:	Programming in Python (CS Major Mid-Term Project)
Group Members:	AIUB Student Group
Development Date:	July 2026

2. Problem Statement & Objectives

Hostel mess facilities in university environments, such as for AIUB students, face significant difficulties in managing meal logging, tracking daily groceries, compiling weekly menus, and reconciling balances. Traditional paper-based registers result in data inaccuracies, disputes, lack of transparent cost auditing, and food waste.

Objectives:

- Establish a secure, role-based login system for Members and Mess Administrators.
- Automate meal ON/OFF scheduling with date constraints to prevent member backdating.
- Implement dynamic, transparent monthly meal rates based on actual grocery costs and meal logs.
- Provide a central approval flow for member feedback and weekly menu change proposals.

3. Feature List & Target Users

Target Users: Hostel Mess Administrators (Provost, House Tutors) and Mess Members.

Core Features:

- **Admin Account Directory:** Create, update, search, and delete (CRUD) member profiles.
- **Interactive Meal Calendar:** View meal participation grids and toggle meal counts.
- **Grocery Ledger:** Track item costs and auto-calculate dynamic monthly meal rates.
- **Ledger Balance:** Log member cash payments, adjust dues, and audit financials.
- **Menu Change Requests:** Submit recommendations (Members) and approve proposals (Admins).

4. Python Concepts Applied (Week 1-5)

- **Variables & Types:** Models map parameters using floats (amounts), strings (IDs), and booleans (meal ON/OFF).
- **Operators:** Calculate meal cost metrics (quantity * unit price) and subtract payments to derive due amounts.
- **Branching (if-elif-else):** Direct users to appropriate role dashboards (Admin/Member) and check field ranges.
- **Loops (for):** Process lists of members, scan entries, and draw days in calendar grids.
- **Functions & Modularity:** Separated logic across specialized modules (main, models, manager, validators, etc.).

5. Data Structures Explanation

- **List:** Used for collections (e.g. `students_list`, `grocery_list`) that require ordering and iterative filtering.
- **Tuple:** Fixed options (e.g. `MEAL_TYPES = ('Breakfast', 'Lunch', 'Dinner')`) to prevent runtime alterations.
- **Set:** Quick index collections (e.g. `student_id_set`, `used_request_ids`) to prevent duplicates and enable $O(1)$ checks.
- **Dictionary:** Configuration keys (e.g. `config.json`) and mapping lookups for menu lists by date.

6. Object-Oriented Design (OOP)

The project follows object-oriented programming. Data models are encapsulated in 6 distinct classes in `models.py`:

- **Student:** Attributes: `student_id`, `name`, `room_number`, `phone`, `join_date`, `role`, `password`, `total_due`.
- **MealRecord:** Attributes: `student_id`, `date`, `breakfast`, `lunch`, `dinner`. Methods: `count_meals()`.
- **GroceryEntry:** Attributes: `entry_id`, `date`, `item_name`, `quantity`, `unit`, `unit_price`, `total_cost`, `added_by`.
- **MessManager:** Centralizes business logic. Manages list buffers, coordinates logins, and saves state to JSON.

7. File & Exception Handling

Data persistence is maintained across 6 JSON databases. The system handles standard edge cases gracefully:

- **FileNotFoundError:** Handled in `file_handler.py`, returning empty structures and creating files on first write.
- **JSONDecodeError:** Catches corrupted/truncated database files, warning the user and initializing empty buffers.
- **ValueError:** Validates input fields, catching non-numeric values and format mismatches without crashing.

8. Library Usage (NumPy & Tkinter)

- **Tkinter (GUI):** Formulates all visual elements, including Sidebar frames, treeviews, form fields, and widgets.
- **NumPy (Math & Stats):** Crucial math operations compiled using NumPy arrays in `calculations.py`:
 - `get_average_daily_cost`: `np.mean()` computes average daily spending.
 - `get_grocery_statistics`: `np.std()` computes standard deviation of expenditures.
 - `get_due_summary`: `np.sum()`, `np.mean()`, and `np.max()` compile member billing balances.

9. Limitations & Future Improvements

- **Database Scaling:** JSON file storage is optimal for mid-term school projects but is not suitable for large concurrent hostel installations. Migrating to SQLite would improve transactional scale.
- **No Real-Time Server Sync:** Since this is a local desktop Tkinter app, it operates on a single file system. Integrating a lightweight REST API would support remote client connections.